

Design Report

ICMP Smurf Attack

A Reflection and Amplification Denial-of-Service Attack

Subsection	A2
Group Number	05
Members	2105040 • 2105039

Report part	Responsible member
1. Overview / definition	2105040 and 2105039 (jointly)
2. Topology, components and timing	2105040 and 2105039 (jointly)
3. Packet and frame structure	2105040 and 2105039 (jointly)
4. Justification of the design	2105040 and 2105039 (jointly)
5. Implementation plan	2105040 and 2105039 (jointly)
6. Expected outcome	2105040 and 2105039 (jointly)
7. Defense idea	2105040 and 2105039 (jointly)

Division of work.

Scope and responsible-conduct statement

This is an academic design document for a network-security course. The Smurf attack is studied so its behaviour can be understood and defended against. It depends on router and host settings switched off by default since 1999 (RFC 2644), so it is effectively extinct on the modern internet. The implementation described here will run only inside a fully isolated laboratory network, on machines the group owns or is authorised to use, with no route to the internet or any campus system, and the attack code will be written by the group from raw sockets rather than taken from any ready-made tool.

1. Overview of the Project Idea

The goal of this project is to reconstruct, in a controlled and isolated laboratory, the classic **ICMP Smurf attack**, to measure how it behaves, and to show why the standard defenses defeat it. The Smurf attack is a **reflection and amplification distributed denial-of-service (DDoS) attack**. That phrase is the heart of the report, so it is worth taking apart in plain terms.

- **Denial of service.** The aim is not to read or steal data but to make a target unreachable, by burying it under more traffic than it can process.
- **Reflection.** The attacker never sends packets straight at the victim. They are bounced off innocent third-party machines, so the victim sees the flood coming from those machines, not the attacker. This hides the attacker.
- **Amplification.** One packet from the attacker becomes many packets at the victim. That ratio is the **amplification factor**, and it lets a weak attacker overwhelm a much stronger victim.

The attack combines three ordinary mechanisms. **IP source-address spoofing**: the attacker forges the packet's "from" field to be the victim's address. **ICMP echo request and reply**: the ordinary ping mechanism, where a host answers an echo request with an echo reply. **IP directed broadcast**: an older router feature that delivers a packet addressed to a subnet's broadcast address to every host on that subnet. Combined: the attacker sends one echo request whose source is forged to the victim and whose destination is a subnet broadcast; the router copies it to every host; every host replies; and because the request looked like it came from the victim, every reply lands on the victim. The attack is named after the 1997 exploit program `smurf.c` that automated it.

2. Network Topology, Components and Timing Diagram

2.1 Topology and components

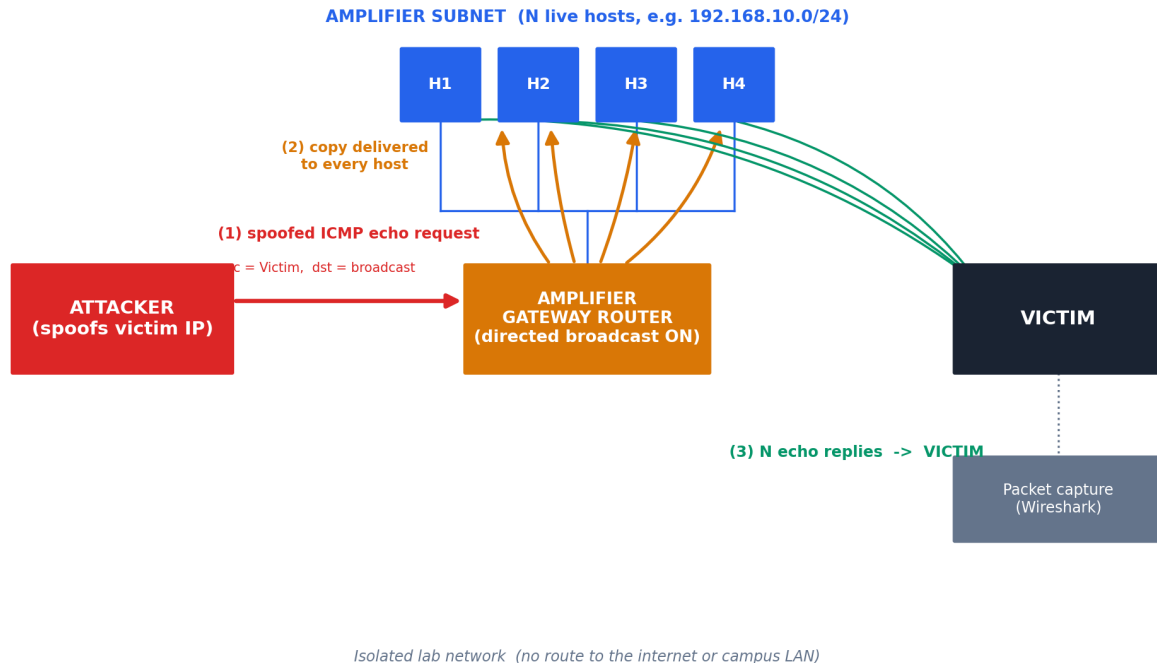


Figure 1: Roles and traffic flow in an isolated lab. One spoofed request (red) is fanned out by the router (amber); every host replies to the forged source, so all replies converge on the victim (green).

- **Attacker.** One machine that crafts and sends the spoofed echo requests; it needs little bandwidth of its own because the amplifier does the multiplying.
- **Amplifier network.** A subnet whose gateway forwards directed broadcasts and which holds several live hosts that answer pings. The more responders, the larger the amplification.
- **Victim.** The target whose address the attacker forges; all replies converge here.
- **Monitoring point.** A capture station beside the victim so the incoming flood can be counted and the amplification factor measured.

2.2 Timing of the original protocol (a normal ping)

To see what the attack changes, start from how ICMP echo is meant to behave. In a normal ping one host sends a single echo request to one other host, which answers with a single echo reply. The exchange is one-to-one, the source address is genuine, and the destination is a single unicast host ("unicast" means addressed to one specific machine, unlike a broadcast that targets a whole subnet). ICMP is connectionless, so there is no handshake.

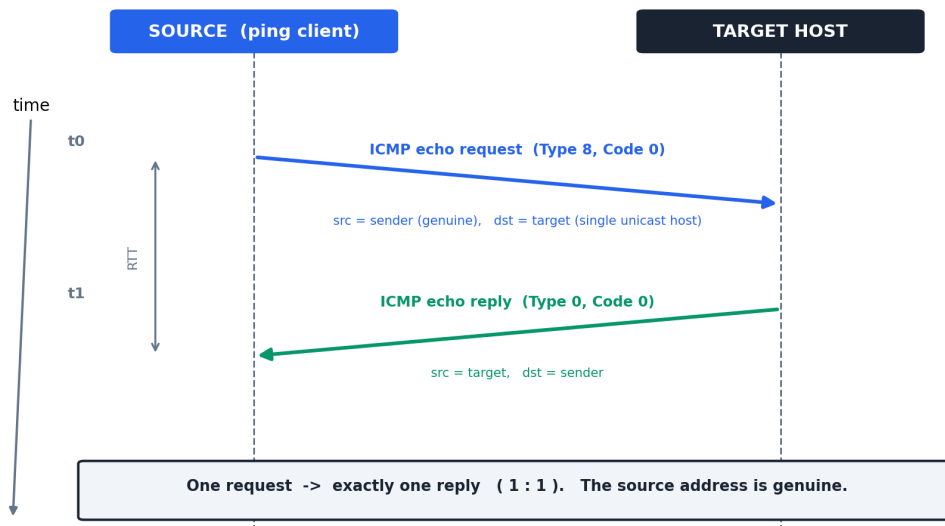


Figure 2: A normal ICMP ping. One request, exactly one reply, truthful source. This 1 : 1 exchange is the baseline the attack subverts.

2.3 Timing of the attack

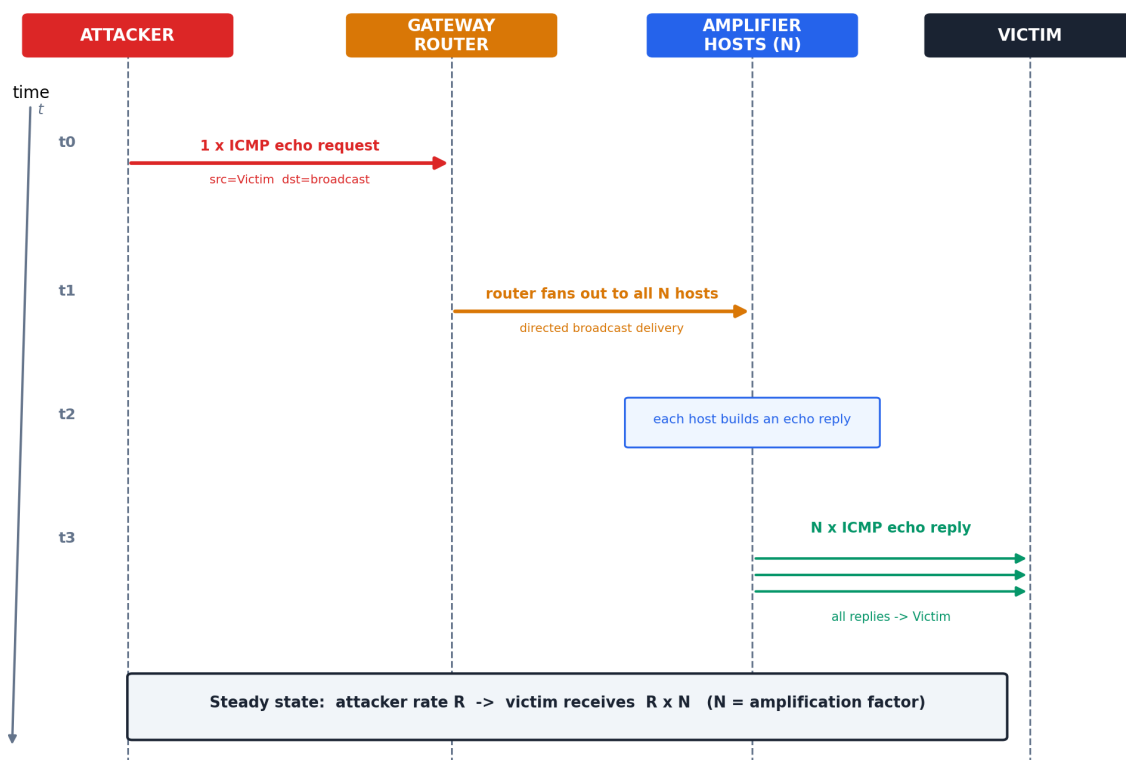


Figure 3: The Smurf attack. The single spoofed request at t_0 becomes N replies at the victim by t_3 ; in steady state, sending at rate R makes the victim receive roughly R times N .

- **t0.** Attacker sends one echo request; source = victim (spoofed), destination = subnet broadcast.
- **t1.** The gateway, with directed broadcast enabled, delivers a copy to every host on the subnet.
- **t2.** Each of the N live hosts builds an echo reply.
- **t3.** Because the request looked like it came from the victim, all N replies go to the victim.

- **Steady state.** At rate R the victim receives about R times N replies per second; on a full /24 subnet N can approach 254.

2.4 Attack strategies

- **Basic single-amplifier flood.** One spoofed request to one subnet broadcast, repeated at a controlled rate; enough to demonstrate both reflection and amplification cleanly.
- **Multiple amplifiers.** Aiming at several subnet broadcasts at once adds each subnet's host count to the total, multiplying the flood further.
- **Bandwidth vs packet-count amplification.** Padding the ICMP payload makes each reply larger, filling the victim's link with bytes faster even when the packet count is fixed.
- **Sustained pressure.** The denial lasts only while traffic keeps arriving, so a real attack streams continuously; in the lab we keep the rate low and instrumented so the multiplier is measurable.
- **Fire-and-forget timing.** ICMP is connectionless, so the attacker never waits for a reply; this is why the attack timing has no handshake.

3. Packet and Frame Structure

Read the attack packet from the outside in, the order a device unwraps it: Ethernet frame, then IP packet, then ICMP message. Only a few fields matter, and those are the ones the attacker changes from a normal ping.

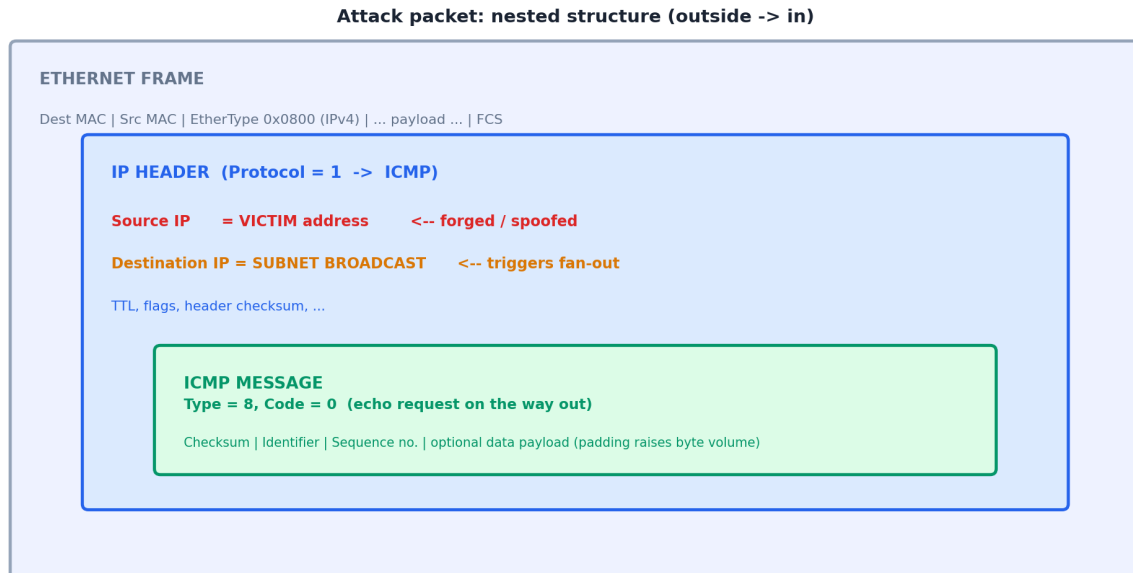


Figure 4: Nested structure of the outbound attack packet. The forged source IP and the broadcast destination IP are the two modified fields that turn a ping into the attack.

Ethernet frame (link-layer wrapper)

Carries the packet across one hop: destination MAC, source MAC, EtherType 0x0800 (IPv4), the IP packet as payload, and a frame check sequence. If we build at the link layer (a layer-2 raw socket) these are ours to fill; if we build at the IP layer the operating system fills them. No link-layer field needs modification for the attack.

IP header (where the modifications live)

- **Source IP = victim's address (MODIFIED).** Normally the sender's own address; forging it to the victim is what makes every reply go to the victim. This is the reflection.
- **Destination IP = subnet broadcast (MODIFIED).** Normally one host; setting it to e.g. 192.168.10.255 makes the router fan the packet out to many hosts. This is the amplification.
- **Protocol = 1.** Unchanged; marks the payload as ICMP.
- **Header checksum.** Because header fields changed, our own code must recompute it or the packet is rejected. The "craft your own packet" rule makes this our responsibility.

ICMP message (unchanged from a normal ping)

- **Outbound:** Type = 8, Code = 0 (echo request), exactly as a normal ping.
- **Inbound (the flood):** Type = 0, Code = 0 (echo reply), one per host, each with the host's own source and the victim's address as destination.
- **Other fields:** an ICMP checksum (also computed by our code), an identifier and sequence number, and an optional data payload we can pad to raise bandwidth amplification.

Attack request versus normal ping and the resulting reply

Field	Normal ping (reference)	Attack request (attacker → amplifier)	Reply (host → victim)
Source IP	Sender's own IP	Victim's IP (forged)	Host's own IP
Destination IP	One unicast host	Subnet broadcast	Victim's IP
ICMP Type/Code	8 / 0	8 / 0 (echo request)	0 / 0 (echo reply)
How many	1 request, 1 reply	1 packet sent	N packets (one per host)

Table 1: The only differences from a normal ping are the two forged address fields; everything else is standard ICMP.

4. Justification: Why the Design Should Work

The design does not depend on luck or on a flaw in one machine. It rests on three behaviours the protocols themselves guarantee, so if the lab conditions are set correctly the amplification and reflection must occur.

1. Every responding host will reply (reflection produces traffic)

RFC 792 specifies that a host receiving an echo request should return an echo reply. This is standard, universally implemented behaviour, so every reachable host willing to answer broadcast pings generates a reply, aimed wherever the request's source field points.

2. A directed broadcast reaches every host (one packet becomes N)

A packet addressed to a subnet's broadcast is by definition meant for every host on it. When the gateway is configured to forward directed broadcasts (the historical setting we re-enable in the lab), it converts that one packet into a link-layer broadcast every host receives, so one request yields one reply per live host. The amplification factor N equals the number of responders, up to about 254 on a /24.

3. IP does not verify the source address (the flood can be aimed)

The IP source field is just data; nothing verifies that a packet came from the address it claims. The amplifier replies to whatever the request claimed as its source, so setting it to the victim is accepted and every reply is delivered to the victim.

Putting it together

The attacker sends one packet, the broadcast turns it into N , and the forged source sends all N to the victim; at rate R the victim receives about R times N . The key property is the **resource asymmetry**: the attacker spends one packet while the victim absorbs N (and the amplifier generates N). Because N can be in the hundreds, a low-bandwidth attacker can saturate a much larger victim. That asymmetry, dictated entirely by the mechanisms above, is why the attack is effective, and it is why we expect a clearly measurable effect in the demo: an amplification factor near the number of live responders, a victim inbound rate near R times N , and rising victim CPU and link use as the flood builds.

5. Implementation Plan

This section is a laboratory methodology. The emphasis is on building the packet ourselves and demonstrating and measuring the amplification, not on raw throughput. In line with the assignment, no ready-made attack tool is used; every header byte and both checksums are produced by our own code.

Step 1: Build an isolated environment

- Use a closed network: several VirtualBox or VMware VMs on an internal-network adapter (a virtual switch with no bridge to the real card), or a few real Linux machines on an isolated switch. Confirm there is no route to the internet or campus network before sending anything.
- Provision the roles: an attacker, a gateway/router, an amplifier subnet with several responder hosts, a victim, and a capture station beside the victim.

Step 2: Deliberately recreate the vulnerable configuration

Because the required behaviours are off by default, the lab must turn them back on on purpose. This is only safe inside the closed network.

- Enable directed-broadcast forwarding on the gateway. On a Linux router this means enabling IP forwarding and setting `net.ipv4.conf.<iface>.bc_forwarding = 1`; on Cisco IOS it is the interface command `ip directed-broadcast`.
- Allow the amplifier hosts to answer broadcast pings by setting `net.ipv4.icmp_echo_ignore_broadcasts = 0` (the opposite of the safe default).
- Leave the path free of egress/anti-spoofing filters so the forged source is not dropped in transit.

Step 3: Write the packet-crafting code ourselves

The whole attack reduces to one carefully addressed packet, sent repeatedly, and we build it with a raw socket so we control every field. In C, this means opening a raw socket, taking control of the IP header, and filling the header bytes directly:

```
s = socket(AF_INET, SOCK_RAW, IPPROTO_RAW); // then set the IP_HDRINCL option
```

- In the IP header we set the source to the victim, the destination to the subnet broadcast, and the protocol to 1 (ICMP), then compute the IP header checksum in our own function.
- In the ICMP header we set Type = 8, Code = 0, fill identifier and sequence, add any payload padding, and compute the ICMP checksum ourselves (the standard 16-bit one's-complement sum).
- We then send the finished buffer to the broadcast address with `sendto()`, in a controlled loop at a chosen rate. A Python version is equivalent: a raw socket, headers assembled with `struct.pack`, and our own checksum routine.
- If we choose to satisfy “frame” literally, we use a layer-2 raw socket (`AF_PACKET`) and build the Ethernet header too; for the attack itself this is optional.

We keep the code to this structural core rather than an optimised flooder, because what we are demonstrating is the packet design and the amplification, which a low, measurable rate shows clearly.

Step 4: Measure the result

- Capture ICMP at the victim (with `tcpdump`, or a short raw-socket sniffer we write) and count packets sent versus received to obtain the observed amplification factor.

- Compare it against the theoretical maximum (the number of live hosts), and record victim CPU load and link utilisation over time.
- Optionally repeat with different host counts and payload sizes to separate packet-count from bandwidth amplification.

Division of implementation work

The report must state which member did each part. A natural split is: one member (e.g. 2105040) writes the raw-socket attack code and checksum routines, and the other (e.g. 2105039) builds the isolated lab, configures the router and hosts, and writes the measurement and defense pieces. The group should confirm and record the final split here.

6. Expected Outcome of a Successful Attack

A successful demonstration produces a set of observable, measurable effects, which are exactly what the measurement step captures.

- **Inbound link saturation at the victim.** The victim's incoming link fills with echo replies; once arriving traffic exceeds capacity, legitimate packets are crowded out.
- **CPU exhaustion at the victim.** Even before the link saturates, processing each reply consumes the victim's CPU and starves its normal work.
- **Loss of legitimate service.** Real users can no longer reach the victim, or face severe slowdowns and timeouts. This is the denial of service.
- **A measurable amplification factor.** The headline result is replies received divided by requests sent, reported next to the count of live responders.
- **Harm to the amplifier as a secondary victim.** The amplifier's own bandwidth and hosts are consumed generating the flood, so a network that permits this abuse hurts itself as well as the target, a strong motivator for the defenses in the next section.

Together these show the defining feature of the attack in numbers: a small, cheap effort at the attacker's end is converted by the amplifier into a disproportionately large, damaging load at the victim's end. If an expected effect fails to appear, it is almost always because a protective default is still in place, which is itself worth reporting.

7. Defense Idea

Smurf is a case study in defenses that succeeded. The measures fall into layers, and the attack is stopped completely if either of the first two is in place.

1. Disable directed broadcast on routers (primary fix)

This removes the amplifier outright: if the router will not fan a broadcast-addressed packet out to every host, one packet stays one packet. **RFC 2644** (1999) changed the recommended default so routers no longer forward directed broadcasts; on Cisco it is `no ip directed-broadcast`, now the default.

2. Anti-spoofing filtering: BCP 38 / RFC 2827

If networks filter outgoing packets whose source address is not their own, the forged “from the victim” packet is dropped before reaching an amplifier. This ingress/egress filtering, documented as **BCP 38 (RFC 2827)**, defeats a whole class of spoofing-based attacks.

3. Host-level suppression

Hosts can refuse to answer pings sent to a broadcast address. On Linux this is `net.ipv4.icmp_echo_ignore_broadcasts = 1`, the modern default. Silent would-be amplifiers produce no flood.

4. Rate limiting and firewall rules

As defence in depth, firewalls and routers can rate-limit ICMP and drop ICMP addressed to broadcast addresses. This blunts impact rather than removing the root cause.

The broader lesson and our planned defense demo

The most important point is **secure-by-default configuration**: Smurf faded not because machines were patched one by one but because the vulnerable behaviours stopped being the default everywhere. A close relative, the **Fraggle attack**, uses the same directed-broadcast amplification with UDP instead of ICMP, and the same two primary defenses stop it, which shows that fixing the misconfiguration matters more than blocking any one protocol. For the bonus, our group plans to implement and demonstrate at least one of these defenses (most directly, disabling directed broadcast on the gateway and enabling host-level suppression) and to re-run the attack to show the flood collapse to nothing, giving a measured before-and-after.